

Lecture 25. Overview of Eigenvalue Algorithms

This and the next five lectures describe some of the classical “direct” algorithms for computing eigenvalues and eigenvectors, as well as a few modern variants. Most of these algorithms proceed in two phases: first, a preliminary reduction from full to structured form; then, an iterative process for the final convergence. This lecture outlines the two-phase approach and explains why it is advantageous.

Shortcomings of Obvious Algorithms

Although eigenvalues and eigenvectors have simple definitions and elegant characterizations, the best ways to compute them are not obvious.

Perhaps the first method one might think of would be to compute the coefficients of the characteristic polynomial and use a rootfinder to extract its roots. Unfortunately, as mentioned in Lecture 15, this strategy is a bad one, because polynomial rootfinding is an ill-conditioned problem in general, even when the underlying eigenvalue problem is well-conditioned. (In fact, polynomial rootfinding is by no means a mainstream topic in scientific computing—precisely because it is so rarely the best way to solve applied problems.)

Another idea would be to take advantage of the fact that the sequence

$$\frac{x}{\|x\|}, \frac{Ax}{\|Ax\|}, \frac{A^2x}{\|A^2x\|}, \frac{A^3x}{\|A^3x\|}, \dots$$

converges, under certain assumptions, to an eigenvector corresponding to the largest eigenvalue of A in absolute value. This method for finding an eigenvector is called *power iteration*. Unfortunately, although power iteration is famous, it is by no means an effective tool for general use. Except for special matrices, it is very slow.

Instead of ideas like these, the best general purpose eigenvalue algorithms are based on a different principle: the computation of an eigenvalue-revealing factorization of A , where the eigenvalues appear as entries of one of the factors. We saw three eigenvalue-revealing factorizations in the last lecture: diagonalization, unitary diagonalization, and unitary triangularization (Schur factorization). In practice, eigenvalues are usually computed by constructing one of these factorizations. Conceptually, what must be done to achieve this is to apply a sequence of transformations to A to introduce zeros in the necessary places, just as in the algorithms we have considered in the preceding lectures of this book. Thus we see that finding eigenvalues ends up rather similar in flavor to solving systems of equations or least squares problems. The algorithms of numerical linear algebra are mainly built upon one technique used over and over again: putting zeros into matrices.

A Fundamental Difficulty

Though the flavors are related, however, a new spice appears in the dish when it comes to computing eigenvalues. What is new is that it would appear that algebraic considerations must preclude the success of any algorithm of this kind.

To see the difficulty, note that just as eigenvalue problems can be reduced to polynomial rootfinding problems, conversely, any polynomial rootfinding problem can be stated as an eigenvalue problem. Suppose we have the monic polynomial

$$p(z) = z^m + a_{m-1}z^{m-1} + \cdots + a_1z + a_0. \quad (25.1)$$

By expanding in minors, it is not hard to verify that $p(z)$ is equal to $(-1)^m$ times the determinant of the $m \times m$ matrix

$$\begin{bmatrix} -z & & & & -a_0 \\ 1 & -z & & & -a_1 \\ & 1 & -z & & -a_2 \\ & & 1 & \ddots & \vdots \\ & & & \ddots & -z & -a_{m-2} \\ & & & & 1 & (-z - a_{m-1}) \end{bmatrix}. \quad (25.2)$$

This means that the roots of p are equal to the eigenvalues of the matrix

$$A = \begin{bmatrix} 0 & & & & -a_0 \\ 1 & 0 & & & -a_1 \\ & 1 & 0 & & -a_2 \\ & & 1 & \ddots & \vdots \\ & & & \ddots & 0 & -a_{m-2} \\ & & & & 1 & -a_{m-1} \end{bmatrix}. \quad (25.3)$$

(We can also get to (25.3) directly, without passing through (25.2), by noting that if z is a root of p , then it follows from (25.1) that $(1, z, z^2, \dots, z^{m-1})$ is a left eigenvector of A with eigenvalue z .) A is called a *companion matrix* corresponding to p .

Now the difficulty is apparent. It is well known that no formula exists for expressing the roots of an arbitrary polynomial, given its coefficients. This impossibility result is one of the crowning achievements of a body of mathematical work carried out by Abel, Galois, and others in the nineteenth century. Abel proved in 1824 that no analogue of the quadratic formula can exist for polynomials of degree 5 or more.

Theorem 25.1. *For any $m \geq 5$, there is a polynomial $p(z)$ of degree m with rational coefficients that has a real root $p(r) = 0$ with the property that r cannot be written using any expression involving rational numbers, addition, subtraction, multiplication, division, and k th roots.*

This theorem implies that even if we could work in exact arithmetic, there could be no computer program that would produce the exact roots of an arbitrary polynomial in a finite number of steps. It follows that the same conclusion applies to the more general problem of computing eigenvalues of matrices.

This does not mean that we cannot write a good eigenvalue solver. It does mean, however, that such a solver cannot be based on the same kind of techniques that we have used so far for solving linear systems. Methods like Householder reflections and Gaussian elimination would solve linear systems of equations exactly in a finite number of steps if they could be implemented in exact arithmetic. By contrast,

Any eigenvalue solver must be iterative.

The goal of an eigenvalue solver is to produce *sequences of numbers that converge rapidly towards eigenvalues*. In this respect eigenvalue computations are more representative of scientific computing than solutions of linear systems of equations; see the Appendix.

The need to iterate may seem discouraging at first, but the algorithms available in this field converge extraordinarily quickly. In most cases it is

possible to compute sequences of numbers that double or triple the numbers of digits of accuracy at every step. Thus, although computing eigenvalues is an “unsolvable” problem in principle, in practice it differs from the solution of linear systems by only a small constant factor, typically closer to 1 than 10. Theoretically speaking, the dependence of the operation count on $\epsilon_{\text{machine}}$ involves terms as weak as $\log(|\log(\epsilon_{\text{machine}})|)$; see Exercise 25.2.

Schur Factorization and Diagonalization

Most of the general purpose eigenvalue algorithms in use today proceed by computing the Schur factorization. We compute a Schur factorization $A = QTQ^*$ by transforming A by a sequence of elementary unitary similarity transformations $X \mapsto Q_j^* X Q_j$, so that the product

$$\underbrace{Q_j^* \cdots Q_2^* Q_1^*}_{Q^*} A \underbrace{Q_1 Q_2 \cdots Q_j}_Q \quad (25.4)$$

converges to an upper-triangular matrix T as $j \rightarrow \infty$.

If A is real but not symmetric, then in general it may have complex eigenvalues in conjugate pairs, in which case its Schur form will be complex. Thus an algorithm that computes the Schur factorization will have to be capable of generating complex outputs from real inputs. This can certainly be done; after all, zerofinders for polynomials with real coefficients have the same property. Alternatively, it is possible to carry out the entire computation in real arithmetic if one computes what is known as a *real Schur factorization*. Here, T is permitted to have 2×2 blocks along the diagonal, one for each complex conjugate pair of eigenvalues. This option is important in practice, and is included in all the software libraries, but we shall not give details here.

On the other hand, suppose A is hermitian. Then $Q_j^* \cdots Q_2^* Q_1^* A Q_1 Q_2 \cdots Q_j$ is also hermitian, and thus the limit of the converging sequence is both triangular and hermitian, hence diagonal. This implies that the same algorithms that compute a unitary triangularization of a general matrix also compute a unitary diagonalization of a hermitian matrix. In practice, this is essentially how the hermitian case is typically handled, although various modifications are introduced to take special advantage of the hermitian structure at each step.

Two Phases of Eigenvalue Computations

Whether or not A is hermitian, the sequence (25.4) is usually split into two phases. In the first phase, a direct method is applied to produce an *upper-Hessenberg* matrix H , that is, a matrix with zeros below the first subdiagonal. In the second phase, an iteration is applied to generate a formally infinite

sequence of Hessenberg matrices that converge to a triangular form. Schematically, the process looks like this:

$$\begin{array}{ccc} \begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \end{bmatrix} & \xrightarrow{\text{Phase 1}} & \begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & & \times & \times & \times \\ & & & \times & \times \end{bmatrix} & \xrightarrow{\text{Phase 2}} & \begin{bmatrix} \times & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & & \times & \times & \times \\ & & & \times & \times \\ & & & & \times \end{bmatrix} \\ A \neq A^* & & H & & T \end{array}$$

The first phase, a direct reduction, requires $O(m^3)$ flops. The second, iterative phase never terminates in principle, and if left to run forever would require an infinite number of flops. However, in practice, convergence to machine precision is achieved in $O(m)$ iterations. Each iteration requires $O(m^2)$ flops, and thus the total work requirement is $O(m^3)$ flops. These figures explain the importance of Phase 1. Without that preliminary step, each iteration of Phase 2 would involve a full matrix, requiring $O(m^3)$ work, and this would bring the total to $O(m^4)$ —or higher, since convergence might also sometimes require more than $O(m)$ iterations.

If A is hermitian, the two-phase approach becomes even faster. The intermediate matrix is now a hermitian Hessenberg matrix, that is, *tridiagonal*. The final result is a hermitian triangular matrix, that is, diagonal, as mentioned above. Schematically:

$$\begin{array}{ccc} \begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \end{bmatrix} & \xrightarrow{\text{Phase 1}} & \begin{bmatrix} \times & \times & & & \\ \times & \times & \times & & \\ & \times & \times & \times & \\ & & \times & \times & \times \\ & & & \times & \times \end{bmatrix} & \xrightarrow{\text{Phase 2}} & \begin{bmatrix} \times & & & & \\ & \times & & & \\ & & \times & & \\ & & & \times & \\ & & & & \times \end{bmatrix} \\ A = A^* & & T & & D \end{array}$$

In this hermitian case we shall see that if only eigenvalues are required (not eigenvectors), then each step of Phase 2 can be carried out with only $O(m)$ flops, bringing the total work estimate for Phase 2 to $O(m^2)$ flops. Thus, for hermitian eigenvalue problems, we are in the paradoxical situation that the “infinite” part of the algorithm is in practice not merely as fast as the “finite” part, but an order of magnitude faster.

Exercises

25.1. (a) Let $A \in \mathbb{C}^{m \times m}$ be tridiagonal and hermitian, with all its sub- and superdiagonal entries nonzero. Prove that the eigenvalues of A are distinct. (Hint: Show that for any $\lambda \in \mathbb{C}$, $A - \lambda I$ has rank at least $m - 1$.)